Missouri State University

Welcome to CSC 365

Internet Programming

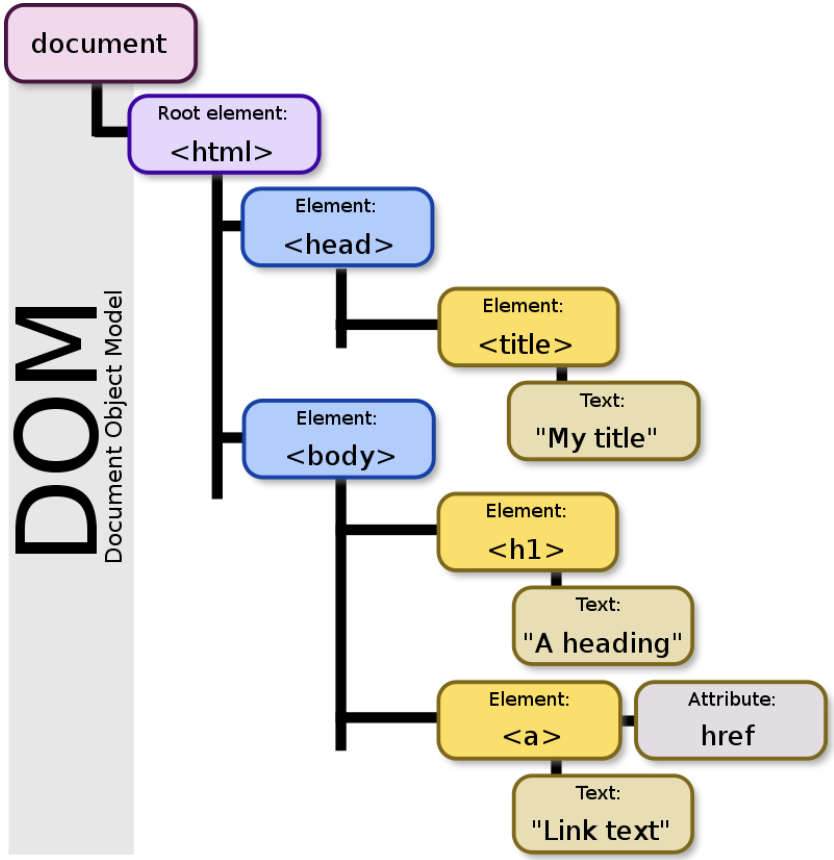
Dr. Yifan Zhang

Learning Objectives

1. Manipulate the DOM via JavaScript.
2. Define a callback function.
3. Understand how asynchronous code executes.
4. Fetch, parse, and use JSON data from an API to populate webpage.

Document Object Model (DOM)

HTML is just a tree, where each element is a node!
We use JavaScript to manipulate this tree.



Document Object Model (DOM)

Rooted at document (html tag)

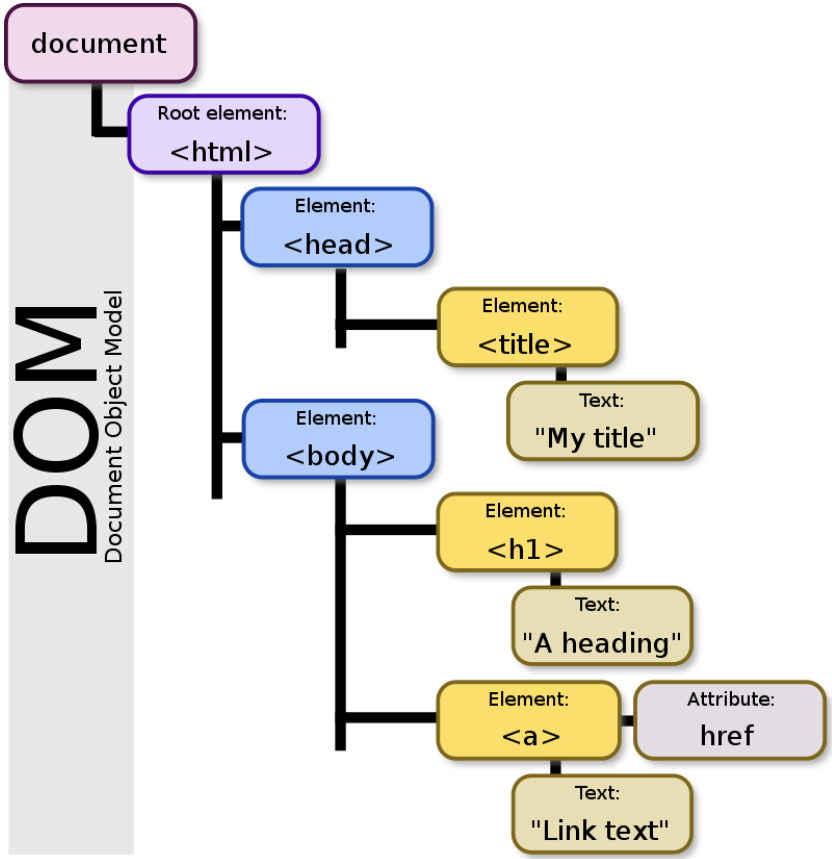
Follows HTML document structure

document.head

document.body

Tree nodes (DOM objects) have tons (~250) of properties, most private

Objects (representing elements, raw text, etc.) have a common set of properties and methods called a DOM "Node"



DOM Node properties and methods

- Identification

  nodeName property is element type (uppercase: P, DIV, etc.) or #comment

- Encode document's hierarchical structure

  parentNode, nextSibling, previousSibling, firstChild, lastChild.

- Provide accessor and mutator methods

  E.g. getAttribute, setAttribute methods, etc..

DOM Node properties and methods

- Walk DOM hierarchy (not recommended)

  element = document.body.firstChild.nextSibling.firstChild;

  element.setAttribute(…

- Use DOM lookup method. An example using ids:

  HTML: .. .

  element = document.getElementById("div42");

  element.setAttribute(…

- Many: getElementsByClassName(), getElementsByTagName(), …
  - Can start lookup at any element

More commonly used Node properties/methods

- textContent - text content of a node and its descendants
- innerHTML - HTML syntax describing the element's descendants.
- outerHTML - similar but includes element " Sample bold display "
- getAttribute()/setAttribute() - Get or set the attribute of an element

```
div.innerHTML = '<strong>Hello</strong> <em>world</em>';
```

Common DOM mutating operations

- Change the content of an element

  element.innerHTML = "This text is important";

  Replaces content but retains attributes. DOM Node structure updated.

- Change an <img> tag src attribute (e.g. toggle appearance on click)

  img.src="newImage.jpg";

- Make element visible or invisible (e.g., for expandable sections, modals)

  Invisible: element.style.display = "none";

  Visible: element.style.display = "";

DOM and CSS interactions

- Can update an element's class

```
element.className = "active";          // replaces all classes
// Better:
element.classList.add("active");        // add one
element.classList.remove("active");     // remove one
element.classList.toggle("active");     // toggle
element.classList.contains("active");   // check
```

- Can update element's style

```
element.style.color = "#ff0000";   // inline style
```

- Can also query DOM by CSS selector

  document.querySelector() and document.querySelectorAll()

Changing the Node structure

- Create a new element (can also cloneNode() an existing one)

```
element = document.createElement("P");
```

or

```
element = document.createTextNode("My Text");
```

- Add it to an existing one

```
parent.appendChild(element);
```

or

```
parent.insertBefore(element, sibling);
```

- Can also remove Nodes: `node.removeChild(oldNode);`
- But, setting innerHTML can be simpler and more efficient.

More DOM operations

- Redirect to a new page

  location.href = "newPage.html";

Note: Can result in JavaScript script execution termination

- Communicating with the user

```
console.log("Reached point A");
console.warn("Be careful!");
console.error("Something went wrong");
```

```
alert("Wow!");           // OK dialog, blocks the page
const ok = confirm("OK?"); // OK/Cancel, returns true/false
// prompt("Your name?");  // input dialog, returns a string or null
```

DOM communicates to JavaScript with Events

Event types:

- Mouse-related: mouse movement, button click, enter/leave element
- Keyboard-related: down, up, press
- Focus-related: focus in, focus out (blur)
- Input field changed, Form submitted
- Timer events
- Miscellaneous:
    - Content of an element has changed
    - Page loaded/unloaded
    - Image loaded
    - Uncaught exception

Event handling

Creating an event handler: must specify 3 things:

- What happened: the event of interest.
- Where it happened: an element of interest.
- What to do: JavaScript to invoke when the event occurs on the element.

Specifying the JavaScript of an Event

- Option #1: in the HTML:

```
<div onclick="gotMouseClick('id42'); gotMouse=true;">...</div>
```

- Option #2: from Javascript using the DOM:

```
element.onclick = mouseClick;
```

or

```
element.addEventListener("click", mouseClick);
```

- Example of the powerful listener/emitter pattern

Event object

- Event listener functions passed an Event object

  Typically, sub-classed MouseEvent, KeyboardEvent, etc.

- Some Event properties:

  type - The name of the event ('click', 'mouseDown', 'keyUp', …)

  timeStamp - The time that the event was created

  currentTarget - Element that listener was registered on

  target - Element that dispatched the event

MouseEvent and KeyboardEvent

- Some MouseEvent properties (prototype inherits from Event)

  button - mouse button that was pressed

  pageX, pageY: mouse position relative to the top-left corner of document

  screenX, screenY: mouse position in screen coordinates

- Some KeyboardEvent properties (prototype inherits from Event)

  keyCode: identifier for the keyboard key that was pressed

  Not necessarily an ASCII character!

  charCode: integer Unicode value corresponding to keypress, if there is one.

document

Use document to reference the DOM.

```
let title = document.getElementById("articleTitle");
let loginBtn = document.getElementsByName("login")[0];
let callouts = document.getElementsByClassName("callout"); // *
```

*class refers to a CSS class

document

Use document to reference the DOM.

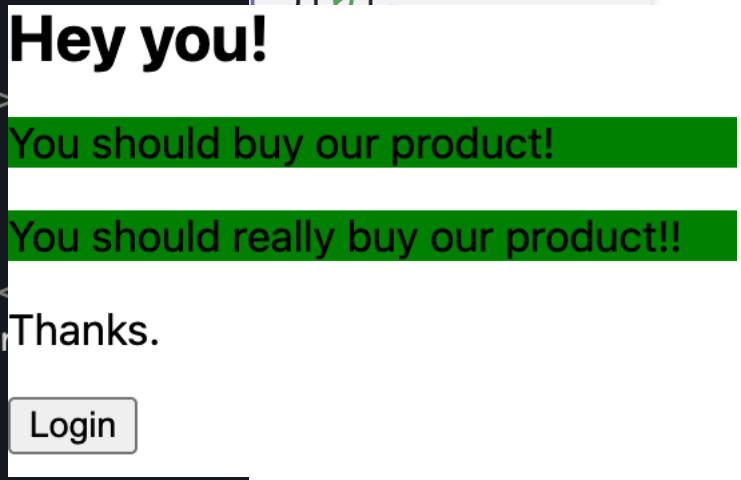
```
<html lang="en">
  <head>
    <link rel="stylesheet" href="styles.css" />
    <script type="module" src="script.js"></script>
  </head>
  <body>
    <h1 id="articleTitle">Hey you!</h1>
    <p class="callout">You should buy our product!</p>
    <p class="callout">You should really buy our product!!</p>
    <p>Thanks.</p>
    <button>Login</button>
  </body>
</html>
```

```
e");
")[0];
"callout"); // *
```

document

Use document to reference the DOM.

```
1  <html lang="en">
2    <head>
3      <link rel="stylesheet" href="styles.css" />
4      <script type="module" src="script.js"></script>
5    </head>
6    <body>
7      <h1 id="articleTitle">Hey you!</h1>
8      <p class="callout">You should buy our product!
9      <p class="callout">You should really buy our pr
10     <p>Thanks.</p>
11     <button>Login</button>
12   </body>
13 </html>
14
```



document

Use document to reference the DOM.

```
console.log('Hello!');

let title = document.getElementById('articleTitle');
let loginBtn = document.getElementsByTagName('button')[0];
let callouts = document.getElementsByClassName('callout');

title.innerText = 'My Website!';
loginBtn.addEventListener('click', () => {
  alert('You are advancing to the next part of the site...');
});

for (let callout of callouts) {
  callout.style.color = 'red';
}
```

document

Use document to reference the DOM.

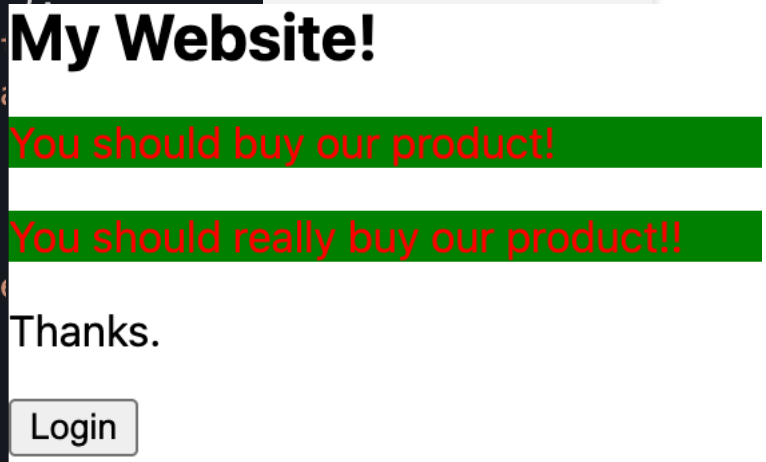
```
let title = document.getElementById("articleTitle");
                                   ")[0];
                                   callout"); // *
```

```
console.log('Hello!');

let title = document.getElementById('articleTitle');
let loginBtn = document.getElementsByTagName('but
let callouts = document.getElementsByClassName('ca

title.innerText = 'My Website!';
loginBtn.addEventListener('click', () => {
  alert('You are advancing to the next part of the
});

for (let callout of callouts) {
  callout.style.color = 'red';
}
```



Object-oriented programming: methods

Using these DOM elements, we can change the title of the article, add an action for when the button is clicked, and make all of the callouts red.

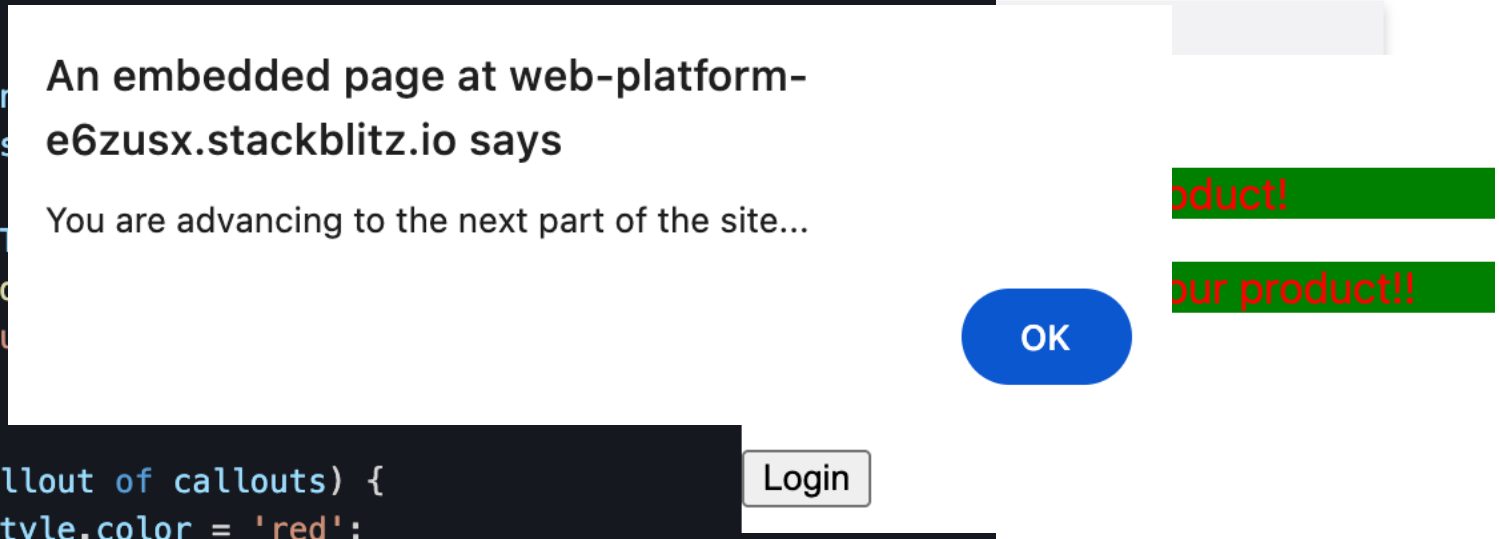
```
title.innerText = 'My Website!';
loginBtn.addEventListener("click", () => {
  alert("You are advancing to the next part of the site...");
});

for (let callout of callouts) {
  callout.style.color = "red";
}
```

Object-oriented programming: methods

Using these DOM elements, we can change the title of the article, add an action for when the button is clicked, and make all of the callouts red.

```
1   console.log('Hello!');
2
3   let title =
4   let loginBtn
5   let callouts
6
7   title.innerT
8   loginBtn.add
9     alert('You
10  });
11
12  for (let callout of callouts) {
13    callout.style.color = 'red';
14  }
```



What is an API?

Definition:
An application programming interface (API)is a set of definitions and protocols for communicationthrough the serialization and de-serialization ofobjects.

JSON is a language-agnostic medium that we canserialize to and de-serialize from!

Refresher: JS Objects

Definition:
Objects are unordered collection of relateddata of primitive or reference types defined using key-value pairs.

```
const instructor = {
  firstName: "Cole",
  lastName: "Nelson",
  roles: ["student", "faculty"]
}
```

JSON and JS Objects

What's the difference?

A JS Object is executable code; JSON is a language-agnostic representation of an object. There are also slight differences in syntax.

```
const drinks = [
  {
    name: "Mimosa",
    ingredients: [
      {name: "Orange Juice", hasAlcohol: false},
      {name: "Champagne", hasAlcohol: true}
    ]
  },
  {
    name: "Vesper Martini", // shaken, not stirred
    ingredients: [
      {name: "Gin", hasAlcohol: true},
      {name: "Vodka", hasAlcohol: true},
      {name: "Dry Vermouth", hasAlcohol: true},
    ]
  }
]
```

Next Class

Week 4:

Topic:

- DOM